Memory and personalisation in personal AI agents

OpenClaw, Hermes Agent, Instinct, and six popular open-source assistants. Same question, two people: how each one makes the answer personal, what it remembers, and how its prompts, skills and tools change over time. With notes on what it means for Gepetel.

Prepared 9 November 2026 from the official OpenClaw docs (docs.openclaw.ai), the Hermes Agent docs (hermes-agent.nousresearch.com) and Honcho's Hermes integration guide. Config names and defaults are as documented on that date. Both projects move fast, so check them against the installed version before relying on a specific default.

> **In one paragraph.** Both agents keep personalisation in **plain files the user can read and edit**, not in model weights. Both split memory into a **small, always-loaded core** (facts, the user profile) and a **large archive searched only when needed** (daily notes or past sessions). Both also learn **procedures** (skills) separately from **facts**.
>
> The difference is in how they keep this up to date:
>
> - **OpenClaw** takes notes freely, then *distils* them in a scheduled "dreaming" sweep. Its core can grow to tens of thousands of characters.
> - **Hermes** keeps a deliberately tiny core (about 3,600 characters in all). It makes the agent curate that core itself, enforced by hard limits, and runs a background review and a skill curator so knowledge doesn't pile up.
>
> Neither changes *which tools exist* as it learns about you. What changes is **what is in the prompt**: memory, profile, the skills index, and recalled context. Tools change only when you or the agent install something (MCP servers, plugins, skills).

---

1. Same question, two people: why the answers differ

Take Ana and Radu, who both send "*What should I cook tonight?*" Neither agent fine-tunes a model per person, so every difference in the two answers comes from **what is put in front of the same model**. Here is what is the same for both, and what isn't.

Input to the model	OpenClaw	Hermes
Base prompt: identity, rules, tool guidance	Same for both: `SOUL.md` and `AGENTS.md` belong to the *agent*	Same for both: `SOUL.md` and tool guidance. Platform hints differ only if they write on different apps.
Tool list	Same, unless policy differs by sender (`toolsBySender`), channel or provider	Same, unless they come in on different platforms (each platform's toolset preset)
Who they are (the profile)	`USER.md`: "Ana is vegetarian, cooks on weeknights in 20 minutes". **Per agent, not per person** (see below).	`USER.md` (built in, **per profile**) and, with Honcho, a **per-person user model** injected every turn
What it knows (facts)	`MEMORY.md`: "fridge has spinach and feta since Sunday"; daily notes found by `memory_search`	`MEMORY.md` snapshot; `session_search` hits ("last time you asked, you liked the lentil curry")

Recall picked for this question	Automatic recall chooses notes relevant to "cook tonight" from *that agent*'s memory	Honcho's context or dialectic answer about *this user* ("prefers quick, spicy, cooks for two kids")
How they like things done (skills)	Skills the agent wrote with that person, such as "weekly meal plan"	Skills created with them, such as "Ana's grocery-list format"
The conversation so far	That session's history (depends on `dmScope`)	That session's history

1.1 The catch: both are personal agents, not multi-user ones

Both systems assume **one agent serves one person**. Personalisation therefore follows the *installation*, not the speaker:

- **OpenClaw: memory is per agent.** `USER.md`, `MEMORY.md` and the daily notes live in the agent's workspace, and memory search covers "only its own configured memory". The docs describe **no per-sender profile inside one agent**, and direct chats share the agent's main session by default. So if Ana and Radu message the same agent, both get *the same* `USER.md`, and in the default shared DM session each may even see the other's messages.
  - The documented fix is **one agent per person**: route each person's DMs to their own agent with a peer binding (`peer: { kind: "direct", id }`), giving each a separate workspace, memory and persona, all behind the same WhatsApp number.
  - Cross-agent session access is on by default (`tools.agentToAgent`) and should be narrowed.
- **Hermes: built-in memory is per profile.** `MEMORY.md` and `USER.md` live in the profile's home folder. The docs don't describe any per-user split within one gateway, so Ana and Radu on the same gateway and profile share one `USER.md`, and their facts mix.
  - Per-person personalisation comes from **Honcho**: each gateway user's platform ID resolves to their own user *peer*, with its own representation, through `userPeerAliases` and `runtimePeerPrefix` (for example `telegram_7654321`).
  - `pinUserPeer: true` collapses everyone into one peer, which is the single-owner setting.
  - Other providers mostly scope by profile or by a single `user_id`, not per person.

> **So the answer to "same question, two people":**
>
> - **Two installs** (the normal case): completely different answers, because every personal file differs.
> - **One OpenClaw agent shared by two people:** the *same* personal context, unless you split them into one agent each.
> - **One Hermes gateway shared by two people:** a shared built-in profile, plus a **per-person Honcho model** if Honcho is on. That makes Honcho the only part of either system designed to tell two people apart inside one agent.
>
> In every case the model, tools and base prompt are the same. Only the injected context differs.
>
> The wider picture (section 7) is the same: of the eight assistants, only the three apps you sign in to (Open WebUI, LobeHub, AnythingLLM) keep memory per user account. Every chat-app agent keeps one memory per agent. Instinct avoids the question by being one account per person.

1.2 How the personal context is used in the answer

Neither system has a separate "personalise this answer" step. The order is:

```
question → assemble context → one model call (or a tool loop) → answer
              │
              ├─ always: profile (USER.md) + facts (MEMORY.md) + skill index
              ├─ this turn: recall for this question (OpenClaw auto-recall / Honcho context)
              └─ on demand: memory_search / session_search / skill_view / honcho_reasoning
```

The model is *told* who it's talking to and does the rest. So the quality of personalisation depends on:

1. **Whether the right fact was written down** (save discipline, the flush, the background review).
2. **Whether it was kept** (dreaming promotion, the Hermes limits, supersession).
3. **Whether it's in front of the model for *this* question** (always-loaded versus recalled versus searched).

Hermes and Honcho add one extra step: `honcho_reasoning` lets the agent *ask a question about the user* ("what would Ana want for dinner on a weeknight?") and get back a synthesised answer, not just stored facts.

1.3 Personalisation in groups

- **OpenClaw:** in a group, `MEMORY.md` is **not loaded**, and each group is its own session. Two people asking the same thing in a group get answers without the owner's private memory. The agent doesn't automatically know who each group member is beyond the conversation.
- **Hermes:** the docs don't describe group handling for built-in memory. With Honcho, third-party write-ups say each participant's messages feed their own peer, but this isn't confirmed in the official docs.

2. How it changes as it gets to know you

The same person, at three points in time. Neither system retrains anything: what changes is the files and stores that feed the prompt, and how much the agent has to look up rather than ask.

When	OpenClaw	Hermes
Day 1	A first-run ritual (`BOOTSTRAP.md`) sets the agent's identity and starts `USER.md`; then the file is deleted. Little is known, so the agent asks. Every message goes into today's daily note when the agent decides it's worth keeping.	`USER.md` and `MEMORY.md` start empty. The agent saves preferences, corrections and "remember this" on its own. A background review after turns adds anything it missed. New saves **don't show in the prompt until the next session** (frozen snapshot).
Week 1	Daily notes pile up (one file a day). The pre-compaction flush saves context before long chats are summarised. The nightly **dreaming** sweep promotes what keeps being recalled, across different questions, into `MEMORY.md`; changed preferences in `USER.md` are marked *superseded*, not deleted. First Workshop skills appear after repeated tasks.	The tiny core fills up: once a write would go past 1,375 characters (profile) or 2,200 (facts), it is **refused** and the agent must merge or drop entries first. So the profile gets *rewritten*, not appended. First skills appear after multi-step tasks, dead ends and corrections. With Honcho, the person's model deepens every turn.
Month 3	`MEMORY.md` is a curated summary of the person; dozens of daily notes sit behind `memory_search`. Risk: past 20,000 characters the core is **truncated** in the prompt (the file stays whole). Skills index grows (~24 tokens per skill, capped). Old skill versions are kept for undo.	Profile and facts are still the same small size, densely rewritten. Every past chat is searchable in SQLite (grows without limit, costs nothing per turn). The **Curator** marks skills unused for 14 days as stale and archives them at 30; an optional pass merges overlapping ones. `/journey` shows the whole timeline.
What visibly changes	Fewer questions (facts are already there), answers using its notes ("you said the fridge has feta"), routines done the way the person likes via skills. Proactive behaviour comes from heartbeat and standing intents, not from memory on its own.	Same, plus a sharper sense of the person through Honcho ("what would they want here?"). The prompt stays the same size, so cost per turn doesn't grow with what it knows.
What's forgotten	Nothing is deleted automatically. Candidates that don't pass the thresholds simply never get promoted; superseded entries stay, marked.	Whatever the agent drops to stay under the limit; archived skills (restorable). Session history is never forgotten.

> **Two opposite bets.** OpenClaw keeps everything and lets a scheduled sweep decide what matters, so knowledge grows and the prompt grows with it. Hermes caps what's always present and makes the agent rewrite it, so knowledge gets denser rather than bigger and the cost per message stays flat.

3. Instinct: the closed-source reference

Instinct publishes nothing about its memory. Everything here comes from one black-box reverse-engineering by Dhravya Shah (founder of Supermemory, a competing memory product), 20 September 2026, done by questioning Instinct over iMessage. Treat the details as **observed or inferred, not documented**.

3.1 What it keeps

- **Git-tracked Markdown files** arranged like a small knowledge graph:
  - `entities/people` and `entities/orgs`;
  - `knowledge/facts`, `knowledge/preferences` and `knowledge/decisions`;
  - conversation digests (`comms/`);
  - `timeline/daily` and `timeline/weekly`;
  - `workstreams/active` and `workstreams/completed`.

  Each record has a header (id, type, **aliases**), then a list of facts, then `[[links]]` to related records.
- Every conversation starts with:
  - an identity block (name, timezone, email);
  - a **memory one-pager of about 4,250 tokens**, with three parts: **life context**, **autonomy calibration** (when to act and when to ask) and **channel communication style**;
  - a to-do index;
  - a recap of earlier conversation of about 9–10k tokens.

3.2 How it writes and reads

- **The answering agent can't write memory.** A separate background job ingests and consolidates, apparently about once a day (one preference took roughly 23 hours to show up as a commit). The one-pager lagged by up to about two days.
- **Consolidation**:
  - moves temporary details into workstreams;
  - shortens records and links them to fuller notes;
  - turns examples into general traits;
  - removes incidental details;
  - replaces wrong facts with **dated corrections**, with old versions kept in git.
- **Retrieval is keyword search, like grep, with no vectors.** "Pasta" finds the dining note, but "Italian noodles I enjoy" or "pazta" find nothing. The aliases exist to make up for that.

3.3 Same question, two people, and change over time

- It's a personal assistant over iMessage, one account per person, so two people get completely separate memories. Group chats aren't covered by the write-up.
- **Over time** the files grow and get consolidated daily, and the one-pager is rewritten from them. Explicit denials were forgotten at ingestion, but time-bound facts ("exams this weekend") lingered. There's **no automatic forgetting**.
- Strengths observed:
  - recalling a single fact;
  - knowing what's recent;
  - handling updates and contradictions;
  - saying when it doesn't know;
  - personalising when asked to.
- Weaknesses observed:
  - **Doesn't personalise unprompted.** "Buy me a monitor" ignored what it knew about his work.

- Struggles to connect facts across chats.
- Has no skills or procedures.

> **Where it sits:** a curated profile always in the prompt, like Hermes; a nightly consolidation that keeps history, like OpenClaw's dreaming; and two things neither open-source agent has.
>
> - **Writing memory is fully separated from answering**, so the agent can't be talked into saving something.
> - **Autonomy calibration is part of the profile.** How much to act alone is learned per person, not set once in the rules for everyone.

---

4. The shared idea: three kinds of knowledge

Kind	What it holds	OpenClaw	Hermes
Who you are profile	Preferences, communication style, relationships, current projects	`USER.md`: dated directives marked active or superseded, with its own 4,000-character cap	`USER.md`: 1,375-character cap (about 500 tokens), the agent's conclusions about you
What it knows facts and decisions	Durable facts, standing decisions, lessons, environment details	`MEMORY.md` (curated) plus `memory/YYYY-MM-DD.md` daily notes (raw)	`MEMORY.md`: 2,200-character cap (about 800 tokens), plus every past session in SQLite
How to do things procedures	Reusable workflows, fixes, "the way that worked"	Skills, including *Skill Workshop* skills the agent writes for itself	Skills (agentskills.io format), created and patched by the agent, pruned by the Curator
Who it is persona	Name, tone, boundaries	`SOUL.md`, `IDENTITY.md`, `AGENTS.md` (operating rules)	`SOUL.md` (or a default identity), plus project context files

The point both designs share: **the model only remembers what was written to disk**. OpenClaw's docs put it as "no hidden state". So personalisation is inspectable, editable and portable. OpenClaw can even import Hermes, Claude Code and Codex memory files.

---

5. OpenClaw

5.1 Where memory lives

Everything sits in the agent's workspace (default `~/.openclaw/workspace`):

- `USER.md` (optional): stable preferences, style, relationships and active projects, written as directives with an observed date and an active or superseded status. When a preference changes, the old entry is *superseded in place* rather than contradicted by a new one.
- `MEMORY.md`: curated long-term facts, decisions and short summaries. It is explicitly *not* a log.
- `memory/YYYY-MM-DD.md`: detailed daily notes and raw working context. They are searchable but not injected every turn.
- `DREAMS.md` (optional): summaries from consolidation, kept for a human to review.

- Bootstrap files: `AGENTS.md` (operating rules, including how to use memory), `SOUL.md` (persona and boundaries), `IDENTITY.md` (name, vibe, emoji). On a brand-new workspace there is also `BOOTSTRAP.md`, a one-time first-run ritual in which the agent sets up its identity and its first model of the user.

5.2 What's in the prompt each session

- `AGENTS.md`, `SOUL.md`, `USER.md` and `MEMORY.md` load at session start. `MEMORY.md` **loads only in the main, private session, never in groups or shared contexts.**
- Today's and yesterday's daily notes are added on a fresh `/new` or `/reset`.
- Budgets: `agents.defaults.bootstrapMaxChars` = 20,000 per file and `bootstrapTotalMaxChars` = 60,000 in total. `USER.md` has its own 4,000 cap. An oversized file is truncated in the prompt but left whole on disk.
- **Automatic recall** can add relevant memory for the task at hand. It has a 1-second preparation budget, and the turn goes ahead without it if it runs late.

5.3 How memory gets written

1. **When asked** ("remember that…"): the agent writes to the right file.
2. **During work:** the generated workspace instructions encourage it to record durable facts.
3. **Pre-compaction flush** (on by default, `agents.defaults.compaction.memoryFlush.enabled`): before a long conversation is summarised, a silent turn on a *private copy* asks the agent to save what matters. Writes append to the daily note only, and replayed whole notes are rejected.
4. **Dreaming** (on by default, `plugins.entries.memory-core.config.dreaming.enabled`): one recurring background sweep.
   - Candidates must pass score, recall-frequency and query-diversity thresholds.
   - A tool-free step decides merges and supersessions.
   - Only "deep promotion" writes to `MEMORY.md`. Summaries go to `DREAMS.md`.
   - **Untrusted and system-derived candidates are never promoted.**

The default heartbeat does no memory maintenance on its own.

5.4 How memory is read back

- `memory_search`: hybrid search, combining vector similarity (OpenAI embeddings by default, local providers possible) with keyword matching over `MEMORY.md` and `memory/**/*.md`. Without embeddings it is keyword-only. It can also surface past session transcripts, which the agent follows up with `sessions_search` and `sessions_history`.
- `memory_get`: reads a specific file or a range of lines.
- `intent`: event-triggered standing intents, such as "when X happens, do Y".
- Engines: the builtin SQLite engine (default), or as plugins **Honcho** (cross-session user modelling) or **LanceDB** (auto-recall and auto-capture). A separate "Memory Wiki" can sit beside either.

5.5 Sessions, people and privacy

- `session.dmScope` defaults to `main`, meaning **all DMs share one session**. The docs warn that with several users this leaks messages between them, and recommend `per-channel-peer`.
- `session.identityLinks` maps one person's identities on different channels onto one peer.
- Groups are isolated per group by default (`session.groupScope: "per-group"`).

- `memory.search.rememberAcrossConversations: true` lets private conversations draw on each other. It is meant for a single trusted owner.
- Resets: none by default. Optional daily reset at `session.reset.atHour` (default 4), optional idle reset, or manual `/new`.

> **Design stance:** OpenClaw is fundamentally **one owner, one agent**. Memory is the owner's memory, and multi-user use is something to configure carefully, not the default.

---

6. Hermes Agent (Nous Research)

6.1 A deliberately tiny, curated core

- `~/.hermes/memories/MEMORY.md`: the agent's own notes (environment, conventions, lessons). `memory_char_limit: 2200`.
- `USER.md`: your profile (preferences, style, expectations). `user_char_limit: 1375`.
- The single `memory` tool has three actions: `add`, `patch` (replace one unique span in an entry) and `remove`. Every call names a target of `memory` or `user`. There is no read action, because the memory is already in the prompt. Exact duplicates are silently rejected.
- **No auto-compaction.** A write that would go over the limit fails, and the agent has to consolidate or remove entries first. This forces curation instead of accumulation.

6.2 The frozen snapshot

Memory and profile are injected into the system prompt **once, at session start**, with usage shown as a percentage. Writes during the session go to disk immediately but **don't change the prompt until the next session** (or a compaction rebuild). This keeps the provider's prompt cache valid.

One side effect: on messaging platforms a chat is one long session, so new memories stay invisible for a long time. The docs recommend `/new` at natural boundaries.

6.3 When it saves

It saves on its own when it learns:

- preferences;
- environment facts;
- corrections;
- conventions;
- completed work;
- anything it was asked to remember.

It skips:

- trivia;
- things that are easy to rediscover;
- raw dumps;
- session-only ephemera.

The docs admit that weaker tool-calling models sometimes *claim* to save without calling the tool.

Background review: after a turn, a review pass may save memory or update skills.

- It is on by default (`auxiliary.background_review.enabled`).
- It uses the main model by default, or a cheaper one working from a digest.
- It can wait until the machine is idle (`defer: auto`).
- A short chat line can announce saves (`display.memory_notifications`).

Approval gates, both off by default:

- `memory.write_approval` stages writes for `/memory approve`.
- `skills.write_approval` stages skill writes for `/skills approve`.

6.4 The archive: every past session

All CLI and messaging sessions go into SQLite (`~/.hermes/state.db`) with FTS5 full-text search. The `session_search` tool returns **real messages, not LLM summaries**, can scroll within a session, and costs no LLM calls. The split is explicit:

- **memory** is for what must always be in context;
- **session search** is for questions like "did we discuss X last week?".

6.5 External user models (Honcho and others)

One external memory provider can run *alongside* the built-in files, never replacing them. The options are RetainDB, ByteRover, Honcho, Hindsight, Supermemory, Mem0, OpenViking and Holographic.

Honcho is the one built for personalisation. It keeps two "peers", a model of **the user** and a model of **the AI**, and builds them from the conversation.

- **What it injects:** durable user context before each response, into the *user message* rather than the system prompt, which keeps the cache intact.
- **Tools:**
  - `honcho_profile`: fast peer card, no LLM.
  - `honcho_search`: hybrid search over past messages.
  - `honcho_context`: summary, representation and recent messages.
  - `honcho_reasoning`: LLM "dialectic" questions about the user.
  - `honcho_conclude`: stores durable conclusions.
- **Settings:** `recallMode` is `hybrid` (default), `context` or `tools`. `writeFrequency` is `async` by default.

6.6 Safety

Memory entries enter the system prompt, so each write is **scanned first**. Prompt-injection patterns, credential-exfiltration patterns, SSH-backdoor patterns and invisible Unicode are all blocked. `/journey` shows a timeline of everything learned (skills and memory) and lets you edit or delete any of it.

> **Design stance:** Hermes treats memory as a **scarce, expensive resource**. Every character sits in every prompt, so the core is small, writes are deliberate, the archive is searched rather than injected, and deeper user modelling goes to an optional external engine.

7. Six more popular open-source assistants

Chosen for everyday use by non-developers, and for both lots of users and lots of contributors (GitHub stars / contributors on 9 Nov 2026). For comparison: OpenClaw 394k / 3.8k and Hermes 262k / 5.4k. Findings come from each project's docs *and* its main-branch source code, read on 9–10 Nov 2026.

Assistant	Stars / contribs	What it is	Two people, same question	Over time
Open WebUI	158k / 893	The most-used self-hosted ChatGPT-style web app	**Per user account.** Each person gets their own memories and their own vector collection, so answers differ.	Memories pile up with no limit and nothing fades. An optional review every 10 turns adds, replaces, moves or removes entries, but only sees the first 80. The injected block is capped at 2,000 + 2,000 characters, so older "about you" entries are silently cut.
LobeHub (LobeChat)	85k / 365	Chat app turning into "super-agents in your team's chats"	**Per user account**, with the richest model: identity, preferences, context, activity, experience, a persona narrative, and a graph of people and things.	A background "Memory Agent" extracts after conversations. Conflicting new facts update old ones, and the persona text is rewritten. Labs adds self-learning: agents propose skills and memory writes, and reviewers, including a nightly one, approve them. What reaches the prompt is bounded by the few most similar memories.
AnythingLLM	68k / 292	Local-first desktop and server app for chat with your documents and agents	**Per user and per workspace.** But its Telegram bot runs everyone as one "null" user, so all Telegram users share memory.	Extracts every 3 hours, once there are 5 or more new chats and 20 minutes of quiet. An Observer picks facts and a Reflector de-duplicates them. **Hard caps of 5 global and 20 per workspace** act as forgetting: once full, new facts are dropped. At most 5 + 5 reach the prompt.
Cherry Studio	53k / 537	Desktop AI workspace that can also put agents in Discord, Slack, Telegram and Feishu	**Per agent.** Everyone talking to the same agent on a channel shares one `USER.md`.	A one-time setup chat writes `SOUL.md` and `USER.md`. After that the agent edits them, rewrites `FACT.md` and appends to a journal. There's no background job, so it changes only when the agent decides. The injected files have no documented cap.

nanobot	50k / 504	Lightweight personal agent on chat apps	**Per workspace**, shared by every chat and sender. The docs: use separate instances for separate people.	**"Dream"** runs every 2 hours. It edits `SOUL`, `USER` and `MEMORY` in place, merges duplicates, replaces corrected facts, archives temporary items after 30 days, and **turns a workflow seen twice into a skill**. Changes are versioned in git, with `/dream-restore` to roll back.
NanoClaw	31k / 186	OpenClaw-style personal agent in containers, **WhatsApp first**	**Per agent group.** The docs: "If different people are involved → separate agent groups", or what's known about one person reaches the other.	One concept per Markdown file, with an index (up to 16k characters) loaded when a fresh context starts. The agent prunes on correction and **can rewrite its own memory rules and standing instructions**. There's no scheduled consolidation, so how tidy it stays depends on the agent.

7.1 How each one puts memory into an answer

- **Always loaded:** Cherry Studio, nanobot and NanoClaw put the whole profile and fact files in the system prompt. They search history only on demand.
- **Picked per message:**
  - Open WebUI loads everything "about you" plus the 8 memories most similar to the last 7 messages. It sends them as a hidden tool result, not in the system prompt.
  - LobeHub loads the persona plus the 3 most similar activities and 3 most similar preferences.
  - AnythingLLM reranks memories against the question and keeps the top 5.
- **Written by:**
  - the model through tools: Open WebUI, LobeHub, Cherry Studio, NanoClaw;
  - a background job: LobeHub (Memory Agent), AnythingLLM (every 3 hours), nanobot (Dream), and Open WebUI optionally.

7.2 What the eight have in common

- **Only three keep memory per user account:** Open WebUI, LobeHub and AnythingLLM. All three are apps you sign in to. Every chat-app agent (OpenClaw, Hermes without Honcho, Cherry Studio, nanobot, NanoClaw) keeps **one memory per agent**, and its docs tell you to run one agent per person.
- **Skills from repetition** exist in nanobot (Dream, after a workflow is seen twice), LobeHub (Labs, reviewed), Hermes and OpenClaw. The three web and desktop apps learn only facts.
- **Explicit forgetting** is rare:
  - AnythingLLM forgets through its hard caps;
  - nanobot archives temporary items after 30 days;
  - Hermes enforces limits and archives skills.

  Everything else grows, and the prompt stays bounded only by truncation or by picking the top few memories.

- **None of the six scans memory writes for injected instructions.** Among all eight, only Hermes does. NanoClaw at least treats all incoming messages as hostile and isolates each group in a container.

8. How prompts, skills and tools change as it learns about you

This is where the two systems actually adapt over time. The short answer: **the tool list is mostly fixed, while the prompt and the skill set evolve.**

8.1 The prompt grows with what it knows

	OpenClaw	Hermes
Always in the prompt	`SOUL.md`, `AGENTS.md`, `USER.md`, `MEMORY.md` (private session only), and the skills index	Three tiers, in order: • **Stable:** `SOUL.md` identity, tool and model guidance. • **Context:** one project file (`.hermes.md`, `AGENTS.md`, `CLAUDE.md` or `.cursorrules`), the git snapshot, platform hints. • **Volatile:** the skills index, the `MEMORY.md` and `USER.md` snapshots, the external provider block, time and session.
How it changes as it learns	New facts reach `USER.md` and `MEMORY.md` directly or through dreaming. The next session's prompt carries them. Superseded preferences are marked, not deleted.	Facts and profile are rewritten by the agent within hard limits. The next session's frozen snapshot carries them. Honcho's injected context changes every turn.
Added per turn	Automatic recall of relevant memory (1-second budget) and search hits the agent asks for	External-provider recall, plugin context and one-line notes when the skill index changed. These go into the user message, so the cached system prompt stays the same.

8.2 Skills: where "learning" really happens

Both agents turn experience into **skills**: folders with a `SKILL.md` and optional files, loaded with **progressive disclosure**. The prompt carries only a compact index (name, description, location). The agent opens the full skill when the task calls for it.

	OpenClaw	Hermes
Index cost	About 24 tokens per skill plus its description, capped by `skills.limits.maxSkillsPromptChars`. Descriptions are shortened first when over the cap.	`skills_list()` is about 3,000 tokens for names and descriptions. `skill_view(name)` loads one skill, and `skill_view(name, path)` one of its files.
Agent writes its own	**Skill Workshop**: saved per agent in `workshop–skills`. Changes apply at once, with the previous version kept for undo.	`skill_manage`: create, patch (preferred), rewrite, add or remove files, delete. Prompted to save after a multi-step workflow worth repeating, a dead end that led to a working path, or a correction from you.

Keeping them good	Previous versions kept; `workshop list/changes/archive/restore`	A linter warns on skills shaped like logs or grown too big. The **Curator** marks skills stale after 14 days unused, archives them after 30, and never deletes on its own. An optional LLM pass merges overlapping skills. Pinned skills and skills used by cron jobs are kept.
Which skills are visible	Gated by what's present: `requires.bins`, `requires.env`, `requires.config`, `os`. Set per agent with `agents.entries.<id>.skills`.	Conditional: `requires_tools` / `requires_toolsets` (shown only when present), `fallback_for_tools` (shown only when the better tool is missing), and `platforms`.
Refresh	File watcher, plus a rescan every 30 minutes. Running sessions pick changes up on their next turn.	The index is frozen for the cache. A one-line note announces a change, and the index itself updates at the next compaction.
Marketplace	ClawHub (`openclaw skills install @owner/slug`), with a verifiable trust envelope	Skills Hub: official, skills.sh, GitHub, ClawHub, LobeHub and others, with trust levels from builtin to community

> **So, as it learns you:** it doesn't gain new *abilities*. It gains **procedures** ("how this person likes their weekly report", "how to book at their dentist's site") that it loads only when the task matches. Over months the skill index becomes a map of your routines, and the curator or archive keeps that map from rotting.

8.3 Tools: mostly fixed, chosen by configuration rather than by learning

OpenClaw's built-in tools:

- **Runtime:** `exec`, `process`, `terminal`, `code_execution`.
- **Files:** `read`, `write`, `edit`, `apply_patch`.
- **Asking the user:** `ask_user`, `secrets` (gets a credential without the model seeing it).
- **Web:** `web_search`, `x_search`, `web_fetch`; **browser:** `browser`.
- **Messaging:** `message`.
- **Sessions and agents:** `sessions_*`, `subagents`, plus goal tools.
- **Scheduling:** `automations` / `cron`.
- **Infrastructure:** `gateway`, `nodes` (paired devices), `plugins`.
- **Media:** `view_image`, image, music and video generation, `tts`.
- **Memory:** `memory_search`, `memory_get`, `intent`.
- **Large catalogues:** `tool_search`, `tool_describe`, `dispatch_action`.

Hermes' built-in tools:

- **Web:** `web_search`, `web_extract`, `x_search` (off by default).
- **Terminal and files:** `terminal`, `process`, `read_file`, `patch`.
- **Browser:** `browser_exec`, `browser_cdp`, `browser_dialog`, `browser_vault`.
- **Media:** `vision_analyze`, `image_generate`, `text_to_speech`.

- **Orchestration:** `todo`, `clarify`, `execute_code`, `delegate_task`.
- **Memory:** `memory`, `session_search`.
- **Skills:** `skills_list`, `skill_view`, `skill_manage`.
- **Scheduling:** `cronjob`.
- **From outside:** MCP tools (one toolset per server, `mcp-<server>`) and plugin tools.

How the tool list is decided:

- **OpenClaw:** policy is applied *before* each model call, in layers: global and per-agent config, channel policy, provider restrictions, sandbox and plugin availability. A tool that is removed sends no schema at all, and `message` narrows itself to the current channel. Profiles, groups and allow/deny lists live on the tool-policy page.
- **Hermes:** tools come in **toolsets** (`web`, `terminal`, `file`, `browser`, `vision`, `skills`, `memory`, `cronjob`, `delegation`, `safe`) bundled into presets per platform (`hermes-cli`, `hermes-telegram`, and so on). They are chosen per session with `--toolsets` or `hermes tools`.
- **Growth** comes from installing MCP servers, plugins or skills, by you, or by the agent where policy allows (OpenClaw's `plugins` tool can install). It doesn't come from learning about you.

8.4 Is the prompt or tool list different for each task?

What varies	OpenClaw	Hermes
By channel or platform	Yes: tool policy per channel, `message` scoped to the channel, `MEMORY.md` left out of groups	Yes: toolset preset per platform, plus platform hints in the prompt
By who is speaking	Yes: `toolsBySender` policy, and session tools can act "as" a user	Per profile (a separate memory per profile)
By model or provider	Yes: `byProvider` tool policy	Yes: tool and model guidance differs, for example extra tool-use enforcement for GPT and Codex
By project or folder	Workspace and project skills	Yes: one project context file, the git snapshot, project skills
By the task in the message	**Partly, at run time:** automatic memory recall picks notes for the task, the agent opens only the skills that match, and `tool_search` loads tool schemas on demand for large catalogues. The *offered* tool list doesn't change per task.	**Partly, at run time:** the agent opens matching skills and searches past sessions, and Honcho injects context relevant to the turn. The system prompt and tool list stay frozen for the session, by design, for caching.
Sub-agents	Delegated runs get their own restricted tool set and sandbox	`delegate_task` sub-agents can skip context files and use the default identity

> **Key point:** neither system routes a task to a different system prompt or tool list up front. Both send **one stable prompt and toolset per session**, kept stable on purpose so the provider's prompt cache keeps working. They adapt **inside** the turn, through skills opened on demand, recall, and (OpenClaw) on-demand tool lookup. The variation that does exist is by *context*, meaning channel, sender, provider and project, not by what the message asks for.

9. Side by side

	OpenClaw	Hermes Agent
Core memory size	Large: up to 20,000 characters per file, 60,000 in total	Tiny: 2,200 + 1,375 characters, hard limits
Archive	Daily Markdown notes, hybrid vector and keyword search	Every session in SQLite FTS5, raw messages, no LLM cost
Who curates	A dreaming sweep that promotes, merges and supersedes	The agent itself (forced by the limits), plus a background review after turns
Changing preferences	Superseded in place, with dates	Patched in place by the agent
Updates reach the prompt	Next session or turn	Next session (frozen snapshot)
Deep user model	Optional Honcho or LanceDB plugin	Optional Honcho (two peers, dialectic) or 7 other providers
Procedural learning	Skill Workshop, versioned	`skill_manage` plus the Curator (stale at 14 days, archived at 30)
Guard on writes	Untrusted candidates never promoted; a flush rejects replays	Injection, exfiltration and Unicode scanning; optional approval queue
Multi-user	Possible but risky by default (shared DM session)	Per profile; mainly single-user
Transparency	Plain files, `DREAMS.md` to review, `openclaw memory status`	Plain files, `/journey` timeline, pending queues

What each gets right, and where it hurts

OpenClaw

- + Captures plenty, distils later, so little is lost.
- + Memory left out of groups by design.
- + Supersession keeps a history of changing preferences.
- – A big core costs tokens on every turn and can drift.
- – The shared DM default is a privacy trap for multi-user bots.
- – Quality depends on a scheduled sweep tuned with thresholds.

Hermes

- + A cheap, cache-friendly core; the limits force good habits.
- + Raw session search is honest (no summary drift).
- + Memory writes are scanned for injection.
- – A frozen snapshot means new facts wait for the next session.
- – Weak models "pretend" to save.
- – The deep user model needs an external service.

10. What this means for Gepetel

The biggest difference is the starting point. Gepetel is **multi-user from the start**: one assistant number, many people. Everything personal is keyed to the person's own 1:1 chat, and group memory is kept apart.

That is exactly where both projects are weakest. OpenClaw needs one agent per person, and Hermes needs Honcho. So for "same question, two people", Gepetel already separates Ana from Radu, and the lessons below are about *depth*, not isolation.

Gepetel is already close to this pattern:

- things kept about each person (`remember_about_them`, the newest 25 injected into every 1:1 reply);
- learned "ways" of doing things per person;
- counts of what works across everyone;
- the getting-to-know-you stage;
- Pulse's record of topics and how each one landed;
- a separate memory on the person's own computer for sensitive details.

Ideas worth taking:

1. **Two tiers, explicitly.** Keep a small always-loaded profile (Hermes size) and move everything else into a searched archive. Gepetel already archives every message, so a `search_past_chats` tool (raw hits, like Hermes' FTS5) would let it answer "what did we say about X?" without growing the prompt.
2. **Supersede, don't append.** Mark a changed preference as superseded with a date (OpenClaw), so "prefers mornings" and "now prefers evenings" can't both be live.
3. **A consolidation pass.** A periodic merge, prune and supersede over each person's memories, like dreaming or the Curator. It must never promote anything that came from an email, web page or group message (OpenClaw's "untrusted candidates are excluded").
4. **Scan memory writes.** Block injection-shaped text and invisible Unicode before anything enters what's kept (Hermes). This matters more now that Pulse reads mail.
5. **Procedures as skills with a lifecycle.** The "ways" are Gepetel's skills: add staleness and archiving (Curator-style) and conditional visibility, so a way that needs the computer is shown only when the computer is paired.
6. **Ask about the person, not just recall facts.** Honcho's "dialectic" step, where the agent asks "what would this person want here?" against everything known about them, is the closest thing either system has to real per-person answers. A cheap version in Gepetel: before answering an open question ("what should I…"), have a small model write two or three lines on what this person would want, from what's kept about them, and put that in the prompt.
7. **Learn autonomy per person** (Instinct): keep "when to act versus ask" as part of each person's profile, learned from how they react, instead of one rule for everyone.
8. **Separate writing from answering** (Instinct and the background extractors): let a background job, not the agent that's answering, decide what gets kept. Then a sentence in an email or a group message can't talk the answering agent into saving it.
9. **A "what I know about you" view** (`/journey`) in the person's 1:1, where they can see, edit and delete what's kept.
10. **Keep tools stable and adapt the context.** Both projects keep one stable toolset and prompt for caching. Gepetel's tool list already varies by context (groups, connectors, computer). This morning's incident, an unrequested group post, is a reminder that a tool being available is not the same as permission to use it; both projects put hard controls in policy, not in memory.

Sources

- OpenClaw: Memory, https://docs.openclaw.ai/concepts/memory
- OpenClaw: Agent workspace, https://docs.openclaw.ai/concepts/agent-workspace
- OpenClaw: Sessions, https://docs.openclaw.ai/concepts/session
- OpenClaw: Skills, https://docs.openclaw.ai/tools/skills
- OpenClaw: Tools, https://docs.openclaw.ai/tools and https://docs.openclaw.ai/gateway/config-tools
- Hermes Agent: Memory, https://hermes-agent.nousresearch.com/docs/user-guide/features/memory
- Hermes Agent: Skills, https://hermes-agent.nousresearch.com/docs/user-guide/features/skills
- Hermes Agent: Curator, https://hermes-agent.nousresearch.com/docs/user-guide/features/curator
- Hermes Agent: Tools, https://hermes-agent.nousresearch.com/docs/user-guide/features/tools
- Hermes Agent: Prompt assembly, https://hermes-agent.nousresearch.com/docs/developer-guide/prompt-assembly
- Honcho: Hermes integration, https://honcho.dev/docs/v3/guides/integrations/hermes
- Open WebUI: https://docs.openwebui.com/features/chat-conversations/memory; source backend/open_webui/utils/memory.py, models/memories.py, config.py
- LobeHub: https://lobehub.com/docs/usage/getting-started/memory; source packages/context-engine/src/providers/UserMemoryInjector.ts, packages/memory-user-memory, packages/builtin-tool-self-iteration
- AnythingLLM: https://docs.anythingllm.com/features/memories; source server/utils/memories, server/jobs/extract-memories.js, server/utils/telegramBot
- Cherry Studio (v2.1.10): docs advanced-basic/agent-workspace/permissions-memory-background.md; source src/main/ai/mcp/servers/agentMemory.ts, src/main/ai/agents/bootstrap.ts
- nanobot: docs/memory.md, docs/concepts.md; source nanobot/agent/memory.py, templates/agent/dream.md
- NanoClaw: docs/memory.md, docs/isolation-model.md, docs/SECURITY.md
- Instinct (reverse-engineered): Dhravya Shah, "Reverse engineering Instinct's memory", 20 Sep 2026, https://supermemory.ai/blog/reverse-engineering-instinct-memory/
- OpenClaw: Multi-agent routing, https://docs.openclaw.ai/concepts/multi-agent
- Hermes Agent: Memory providers, https://hermes-agent.nousresearch.com/docs/user-guide/features/memory-providers

Where the documentation contradicts itself, this report follows the configuration reference:

- Hermes' curator page also mentions 30 and 90 days unused, against the 14- and 30-day defaults.
- Its prompt-assembly page places the skills index and the platform hints in different tiers in different sections.